SI7011 · Evento evaluativo 2 · Parte A: leer la evidencia

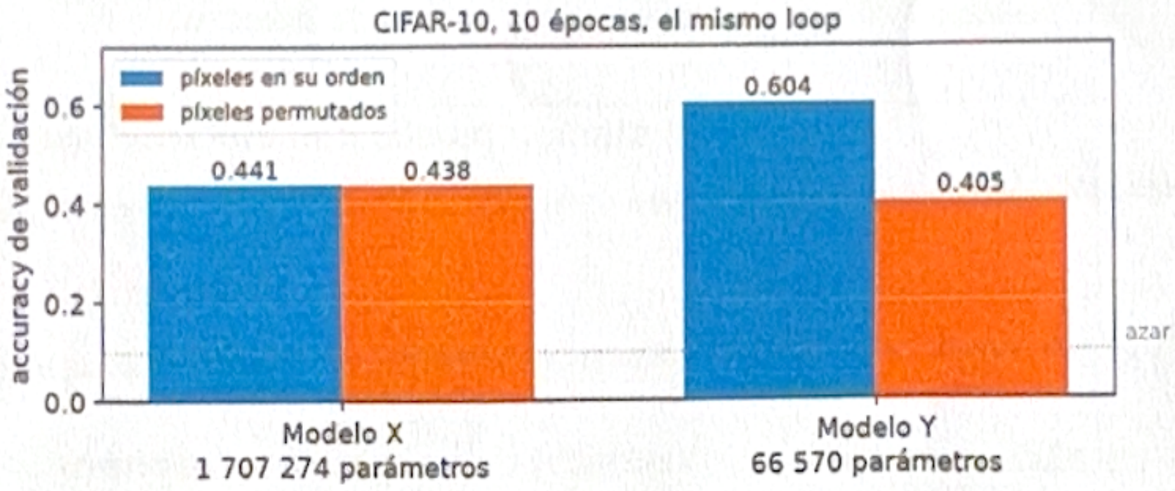
Nombre: Andrés Felipe Agudelo Ortega / Israel Garnica Bohórquez **Documento:** 1037672497 / 910803 47

Individual · sin computador ni apuntes · 30 minutos · 6 preguntas, 1 punto cada una.
Respuestas cortas: una o dos frases por literal. Cuando la pregunta pide evidencia, cita un número leído de la figura o la tabla. Todos los resultados son de corridas reales de los notebooks de la sesión 03.

---

1. Permutar los píxeles

Uno de los modelos es un MLP sobre la imagen aplanada y el otro una CNN pequeña. «Permutados» quiere decir que se baraja el orden de los píxeles con **una misma permutación fija** para todas las imágenes (entrenamiento y validación) y se entrena de nuevo.



a) ¿Cuál es el MLP y cuál la CNN? Justifica con dos números de la figura.

El modelo X es el MLP, por que por su arquitectura, no diferencia entre imágenes con y sin permutación; su precisión se mantiene. El modelo Y es la CNN, ya que al permutar los pixeles, pierde considerablemente su precisión; CNN depende del orden para que el accuracy sea bueno.

b) ¿Por qué la permutación casi no afecta a uno y sí al otro? Nombra la suposición sobre la imagen que se rompe.

Por que la CNN aprende identificando patrones entre pixeles vecinos (mediante kernels). Al desordenarlos, se pierde la relación espacial. En cambio, el MLP trata cada pixel como una entrada independiente. Si desordeno la imagen, el MLP no tiene la capacidad de saber que se desorganizo, la interpretará de la misma forma.

2. Formas, parámetros y campo receptivo

Una imagen de EuroSAT de $3 \times 64 \times 64$ entra a:

```
Conv2d(3, 32, kernel_size=3, padding=1) → ReLU →
Conv2d(32, 64, kernel_size=3, padding=1, stride=2) → ReLU → MaxPool2d(2)
```

a) Escribe la forma $C \times H \times W$ de la salida de cada una de las tres capas.

1er CNN → 32 x 64 x 64
2do CNN → 64 x 32 x 32
MaxPool → 64 x 16 x 16

La 1era CNN aumenta el tamaño de la imagen, integrando los canales RGB; la 2da CNN reduce la anterior a la mitad por el stride; y MaxPool vuelve a reducir a la mitad.

b) ¿Cuántos parámetros tiene la segunda convolución? ¿Cuántos tendría una capa densa con la misma entrada y la misma salida? ¿Qué dos propiedades de la convolución explican la diferencia?

$64 \cdot (32 \cdot 3 \cdot 3) + 64 = 18\,496$ Parámetros 2da CNN
(filtros · canales entrada · kernel size · bias)

$n_{in} \cdot n_{out} + n_{out} \cong 8{,}59 \times 10^9$ parámetros de capa densa

$(32 \times 64 \times 64) \cdot (64 \times 32 \times 32) + 65536$
131072 65536

Propiedad 1: CNN reduce Conexiones a solo la ventana del kernel.
Propiedad 2: los pesos del kernel se comparten.

c) ¿Qué región de la imagen de entrada ve una posición a la salida de la segunda convolución? ¿Y a la salida del `MaxPool2d`?

$r_l = r_{l-1} + (k_l - 1) \prod_{i<l} s_i$

donde $r_0 = 1$, originalmente 1 solo pixel

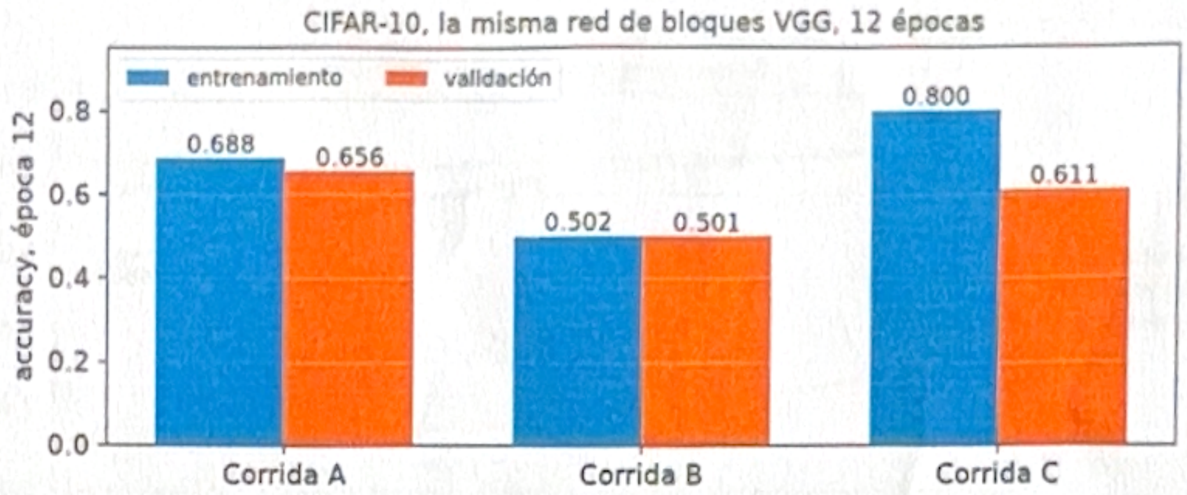
1er CNN K=3 / S=1.
$r_1 = 1 + (3-1)(1)$
$r_1 = 3$
región de 3x3 pixeles

2da CNN K=3 / S=2
$r_2 = 3 + (3-1)(1)$
$r_2 = 5$
región de 5x5 pixels.

3ra Maxpool K=2 / S=2.
$r_3 = 5 + (2-1)(1 \times 2)$
$r_3 = 7$. región de 7x7 pixels

3. BatchNorm y data augmentation

Las tres corridas usan la misma red de bloques VGG: una **sin** BatchNorm, otra **con** BatchNorm, y otra con BatchNorm **y** data augmentation (recortes y flip horizontal).



a) Asigna A, B y C. Justifica cada una con un número de la figura.

A → BatchNorm + data augmentation → mejor resultado en validación (0,656)

B → Sin BN → La peor validación (0,501)

C → BN sin augmentation → El mejor entrenamiento (0,8) pero baja en validación (0,611)

b) Una corrida tiene **menos** accuracy de entrenamiento que otra pero **más** en validación. Explica por qué, y di qué número calcularías para mostrarlo.

Porque el data augmentation presenta imágenes modificadas que incrementan dificultad en entrenamiento, pero ayudan a reconocer imágenes nuevas. Para demostrarlo, calcularíamos diferencia de accuracy entre entrenamiento y validación, por ejemplo 0,032 → A; 0,001 → B; 0,189 → C; En este caso vamos a A comparado con C, donde A tiene menor entrenamiento que C Pero su diff con validación es mucho menor

4. Profundidad y atajos

Dos redes de 20 capas convolucionales con BatchNorm, sobre CIFAR-10: las mismas convoluciones, la misma inicialización, los mismos datos y la misma augmentation, 8 épocas. Una tiene atajos residuales y la otra no.

	parámetros	pérdida de entrenamiento	acc. entrenamiento	acc. validación
Red P	269 722	1.189	0.565	0.529
Red Q	272 474	0.992	0.639	0.606

a) Un compañero dice: «la red P sobreajusta». ¿Estás de acuerdo? Justifica con dos números de la tabla.

con los datos presentados, no hay evidencia clara de sobreajuste. La red P tiene accuracy de 0,565 y 0,529 en entrenamiento y validación, son valores bajos y cercanos. Puede que tenga dificultad durante aprendizaje.

b) ¿Cuál red tiene atajos? Escribe la salida de un bloque residual en función de su entrada a_l y explica por qué el atajo ayuda a entrenar una red profunda.

La red Q tiene atajos, al demostrar un mejor accuracy en validación (0,606 vs 0,529). Como un bloque residual calcula $a_{l+1} = F(a_l) + a_l$ el atajo permite que la información original se contraste con las 20 capas en el paso final.; Adicionalmente, la red Q posee un mayor número de parametros que la red P, esto se debe a los conexions residuales cuando hacen ajuste dimensional

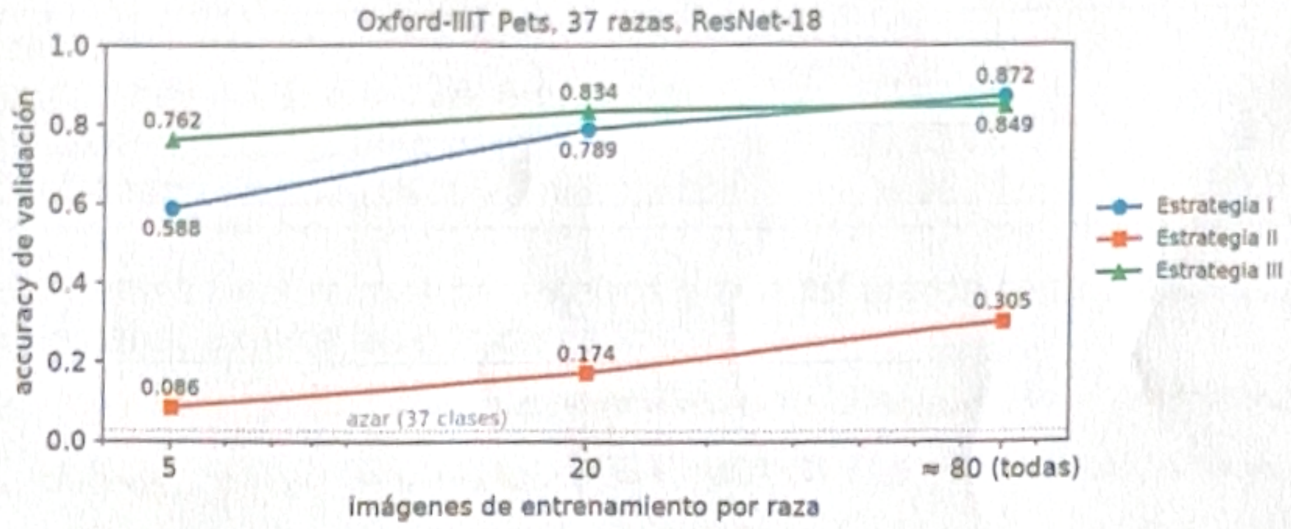
c) Q tiene 2 752 parámetros más que P. ¿De dónde salen, si los atajos suelen ser la identidad?

Los 2752 parámetros corresponden a los pesos adicionales de los atajos que ajustan dimensiones en la identidad; Si suponemos convolución de 1×1, 43 canales de entrada y 64 filtros de salida: $P = (C_{in} + 1) \cdot C_{out} \Rightarrow (43 + 1) \times 64 = 2752$ para el atajo residual.

1
Sesgo

5. Tres estrategias de transfer learning

Una ResNet-18 sobre Oxford-IIIT Pets (37 razas) con tres estrategias: entrenar **desde cero**, *linear probe* sobre el backbone de ImageNet **congelado**, y *fine-tuning* de toda la red partiendo de ImageNet.



a) Asigna I, II y III. Justifica cada una con un número de la figura.

I-Azul → Fine-tuning → mejora de 0,588 a 0,872 al aumentar los datos desde imagenet.

II → Naranja → Entrenamiento desde 0 → peor accuracy (0,305) con todas las imagenes.

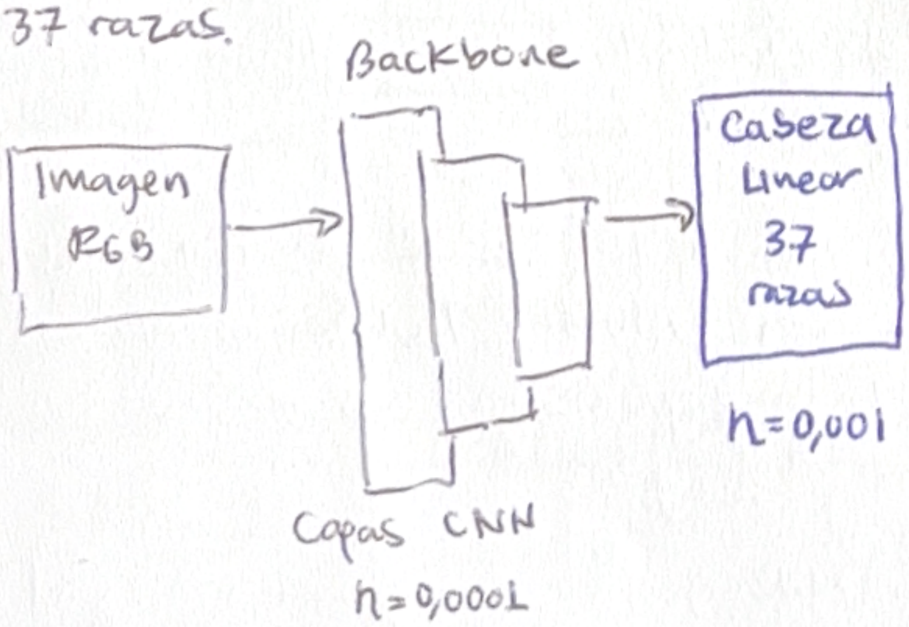
III → Verde → Linear Probe → mejor resultado inicial (0,762 con solo 5 imágenes x raza) aprovechando aprendizaje previo.

b) Con 5 imágenes por raza, I queda por debajo de III; con todas, la supera. Explica por qué.

Con pocos datos, III funciona mejor por que utiliza conocimiento previo aprendido, debido al backbone congelado, pero empeora en su adaptación a nuevas imágenes. En cambio, I puede actualizar los pesos del backbone. Con pocos datos, se perjudica, pero al aumentar los nuevos datos, se adapta mejor y supera levemente (0,872 vs 0,849).

c) En el fine-tuning, el backbone usa $\eta = 10^{-4}$ y la cabeza nueva $\eta = 10^{-3}$. ¿Por qué no usar el mismo η para toda la red?

Se utiliza una tasa pequeña en el backbone para conservar lo aprendido previamente, y una tasa mayor en la cabeza nueva para que aprenda más rápido a identificar las 37 razas.



Si se pusiera un η mas grande en el backbone puede que desordene los pesos y se perderia lo aprendido en image net

6. CNN frente a ViT

CIFAR-10, 50 000 imágenes de entrenamiento. Las tres primeras filas se entrenan desde cero con el mismo loop durante 20 épocas.

Modelo	parámetros	acc. validación	tiempo
CNN (bloques VGG + BatchNorm)	72 954	0.693	21 s
ViT pequeño (patches 4×4, $d = 128$, 6 bloques)	811 146	0.603	75 s
El mismo ViT sin embeddings de posición	811 146	0.551	78 s
ViT-B/16 preentrenado en ImageNet + *linear probe*	≈ 86 M	0.943	—

a) El ViT pequeño tiene 11 veces más parámetros y aun así pierde contra la CNN, pero el ViT-B/16 preentrenado le gana. Explica las dos cosas.

La CNN supera ViT pequeño por que aprovecha mejor los patrones de las imágenes y requiere menos datos para aprender. El ViT-B/16 alcanza mayor precisión (0,943) por el preentrenamiento, pero al no reportar el tiempo y con 86 millones de parámetros, se espera un mayor costo de procesamiento.

b) Sin embeddings de posición, ¿qué información pierde el modelo? ¿Por qué aun así queda muy por encima del azar (0.10)?

Sin embeddings de posición, el ViT pierde información sobre la ubicación de los fragmentos de la imagen, reduciendo su accuracy (0,551). Sin embargo, sigue reconociendo patrones visuales dentro de los fragmentos, superando al azar (91).

c) Con imágenes de 32×32 y patches de 4×4, ¿cuántos tokens entran al Transformer, contando [CLS]? Con imágenes de 64×64 y los mismos patches, ¿cuántos, y por qué factor crece la matriz de atención?

Una imagen de 32×32 genera 65 tokens; mientras que una imagen de 64×64 genera 257 tokens, incluyendo [CLS]. La matriz de atención crece ≥ 16 veces, porque cada token se compara con todas las demás.

